ASSIGNMENT 19
SQL Injection Testing of an Android Content Provider

1. Introduction

Android Content Providers offer a standard interface for sharing structured data between applications. They expose data through content URIs and operations such as query, insert, update, and delete. If a provider builds database statements unsafely or exposes data without appropriate access controls, a caller may be able to retrieve or modify records beyond the intended scope. This practical focuses on reviewing a provider in an authorized test app and evaluating the risk of SQL Injection and unauthorized data access.

2. Aim

To assess whether an Android Content Provider safely handles query parameters and enforces access controls, and to recommend measures that prevent injection and unintended data exposure.

3. Background

A Content Provider may receive selection clauses and selection arguments through its query interface. Safe implementations bind user-controlled values as arguments and avoid accepting arbitrary SQL fragments from callers. Provider permissions, URI path validation, and exported-component settings also determine which applications can reach the data.

SQL Injection is only one possible weakness. A provider can still expose sensitive records even when its SQL is parameterized if it allows untrusted callers to query unrestricted columns or rows.

4. Tools Required

- Android Studio and an emulator or test device.
- A purpose-built Content Provider test application with dummy records.
- Android Debug Bridge (ADB) for inspecting the test device.
- AndroidManifest.xml and provider source code for review.
- A worksheet for recording query behavior, permissions, and data exposure.

5. Testing Procedure

Step 1: Identify the provider

Review AndroidManifest.xml and locate the Content Provider declaration. Record its authority, whether it is exported, and any read or write permissions. Confirm that the application is a lab target that you own or have explicit permission to test.

Step 2: Map the exposed data

Inspect the provider implementation and document the content URIs, tables, columns, and operations supported. Identify whether the provider returns personal or confidential fields, and use only synthetic test records.

Step 3: Review query construction

Locate the provider's query() method and check how it processes the URI, projection, selection, selection arguments, and sort order. Flag code that concatenates caller-controlled text into SQL or passes unrestricted SQL fragments to the database.

Step 4: Run baseline queries

From the app's own test screen or an authorized ADB-based test, issue a normal query that should return a known dummy record. Record the expected columns and number of rows. Then run a query for a non-existent record and confirm that no unrelated rows are returned.

Step 5: Check input handling in the lab

Use a controlled test value containing SQL punctuation against the local test provider. Do not use destructive statements or test another application's provider without authorization. Observe whether the provider treats the value as a literal argument, rejects it, produces an error, or returns records outside the expected scope. Record only the behavior observed.

Step 6: Check access controls

Verify that an unrelated test application cannot read provider data unless access is intentionally granted. Review exported status, read/write permissions, URI permission grants, and whether the provider limits results to the requested record. A correctly parameterized query does not compensate for missing authorization.

Step 7: Remediate and retest

Use selection arguments for values, validate URI paths and requested columns, avoid accepting arbitrary SQL clauses from external callers, and enforce least-privilege provider permissions. Repeat the baseline and controlled-input tests to confirm that inputs do not alter query logic and unauthorized callers cannot retrieve protected records.

6. Example of Safe Query Handling

The following Android SQLite pattern binds the record identifier separately from the SQL statement:

```
Cursor cursor = db.query("documents",
    new String[] { "_id", "title" },
    "_id = ?", new String[] { documentId },
    null, null, null);
```

The question mark is a placeholder, and documentId is supplied as a value. The provider should also verify that the caller is permitted to access the requested document and close the cursor when finished.

7. Risk Analysis

- Confidentiality: exposed records may reveal names, messages, documents, or other stored information.
- Integrity: if unsafe update or delete operations are reachable, records could be changed or removed.
- Cross-application exposure: an exported provider with weak permissions may allow another installed app to access data.
- Privacy and compliance: disclosure of personal data can create user harm and regulatory obligations.
- False confidence: parameterized SQL prevents injection into values, but does not automatically enforce row-level authorization or safe provider configuration.
-

8. Observation Table

Test performed	Expected secure result	Risk indicator	Recommended action
Normal URI query for a dummy record	Only the requested record is returned	Extra unrelated rows suggest weak filtering	Bind values and constrain the query
Query with SQL punctuation in a value	Input is treated as literal data or rejected	Result set changes unexpectedly	Use selectionArgs and validate input
Request for a non-existent record	No rows returned	Other records are returned	Review selection logic and URI mapping
Access from an unauthorized test app	Access denied unless explicitly permitted	Provider data is readable without permission	Restrict exported access and add permissions
Attempt to request unapproved columns	Request is rejected or restricted	Sensitive columns are exposed	Use a projection allowlist

9. Result

The Content Provider was assessed for unsafe SQL construction, input handling, exposed data, and access-control weaknesses. A secure provider should treat user-controlled values as data, limit results to authorized records, and reject unauthorized access. Enter actual outcomes from the approved test environment before submitting the practical.

10. Precautions

- Test only a provider in an application you own or are explicitly authorized to assess.
- Use dummy records and avoid personal or production data.
- Do not run destructive update or delete tests on real data.
- Do not assume a provider is safe solely because it uses parameterized queries; review permissions and row-level access.
- Record errors and unexpected results without exposing confidential data.

11. Conclusion

Content Provider security depends on both safe database operations and appropriate access control. SQL Injection may occur when caller-controlled input is inserted into SQL syntax, while weak provider permissions can expose data even without injection. Parameterized values, strict URI and column validation, least-privilege permissions, and focused retesting help reduce these risks.

12. References

- Android Developers documentation: Content Provider basics and provider permissions.
- Android Developers documentation: SQLiteDatabase query methods and selection arguments.
- OWASP Mobile Application Security Testing Guide (MASTG).